Q3: Databases, Room, DAO, Firestore and Data Synchronization

** = supplementary, not in the lecture materials · Persistence · storage options · SQL with SQLite · Kotlin data classes · Room (Entity, DAO, Database) · Cloud Firestore · sync strategies · comparisons

1. Why data persistence is needed

Persistence means saving data so it **survives** after the app is closed, after the process is killed, after a device restart, and after configuration changes such as **screen rotation**. Without persistence, data disappears when the screen or process is recreated. With persistence, data is restored when the app opens again.

Typical needs: remembering a logged-in user, storing registered users, showing a list of students.

Design goal: choose storage based on data **size**, **structure**, **sharing** (one device or many users) and **connectivity** (must it work offline?).

2. Android storage options

Storage	What it stores	Use for	Limitation
SharedPreferences	Small **key-value** pairs	Settings, session flags (“isLoggedIn”), theme choice	No tables, no relations, no SQL queries; unsuitable for large lists
SQLite	Structured **relational tables**, queried with **SQL**, stored **locally** on the device	Tables, relationships, filtering	Manual SQL, Cursor handling, more boilerplate
Room	Same SQLite database, accessed through a modern Jetpack **abstraction**	Structured local data in modern apps (**preferred**)	Still local only; needs separate sync for other devices
Cloud (Firebase Firestore / Realtime Database)	Remote data on a server	Data **shared across devices and users**	Needs network, security rules, sync and conflict handling
Hybrid (Room + cloud)	Local cache plus cloud source	Offline-first apps	Must handle sync, conflicts, stale data

Key decision question: Is the data only for this device, or must it be available across devices and users?

Local vs Cloud vs Hybrid (lecture comparison)

Local database	Cloud database	Hybrid approach
Stored on the device	Stored remotely	Local cache + cloud source
Fast local access	Shared across devices	Offline-first experience
Works offline	Centralised data	Synchronization required
Device-specific unless synchronized	Network dependency and security rules	Must handle conflicts and stale data

Decision guide (lecture)

Situation	Choose
Small settings / flags	SharedPreferences
Structured local data	Room (over SQLite)
Shared remote data	Firebase Firestore / Realtime Database
Offline + cloud	Room local cache + Firebase + a synchronization strategy

Questions to ask: What is the data structure? Must it work offline? Must multiple devices/users see the same data? How often does it change? What happens when two changes conflict?

3. SQLite

SQLite is an **embedded relational database** built into Android. “Embedded” means it runs inside the app (no separate server). It uses **SQL** syntax and stores the database **file locally** on the device.

- Advantage over SharedPreferences: structured **tables + SQL querying + local persistence**.
- Use it when an app needs tables, relationships, filtering and SQL queries.

SQLite architecture in Android

Application → SQLiteOpenHelper → SQLiteDatabase → Tables / SQL

Layer	Role
SQLiteOpenHelper	Manages **database creation** and **version upgrades**
SQLiteDatabase	Gives readable/writable access (`readableDatabase`, `writableDatabase`) and **executes SQL operations** against tables
Tables / SQL	Where the data actually lives

Creating the database helper (lecture code)

```
class DBHelper(context: Context) :
    SQLiteOpenHelper(context, "UserDB", null, 1) {   // (context, db name, cursor factory, version)

    // Runs ONCE, the first time the database is created
    override fun onCreate(db: SQLiteDatabase) {
        val sql = """
            CREATE TABLE users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT, email TEXT, password TEXT
            )
        """
        db.execSQL(sql)
    }

    // Runs when the version number is increased (schema change)
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS users")
        onCreate(db)
    }
}
```

onCreate vs onUpgrade: `onCreate()` creates the tables the first time the database is made. `onUpgrade()` runs when you increase the database version, to change the schema (here it drops and recreates the table; a real app would migrate, for example with `ALTER TABLE`, to keep existing rows).

4. SQL queries (learn the exact keywords)

SQLite data types **: `INTEGER`, `TEXT`, `REAL` (decimal), `BLOB` (binary), `NULL`. Constraints: `PRIMARY KEY` (unique row ID), `AUTOINCREMENT` (ID generated automatically), `NOT NULL` (value required), `UNIQUE` (no duplicates), `DEFAULT value`.

Example table used below: **students(id, name, regNo, department, gpa)**

```
-- Create a table
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    regNo TEXT UNIQUE,
    department TEXT,
    gpa REAL
);

-- CREATE (insert a row)
INSERT INTO students (name, regNo, department, gpa)
VALUES ('Nimal', 'IT21001', 'IT', 3.4);

-- READ
SELECT * FROM students;                                  -- all rows, all columns
SELECT name, gpa FROM students;                          -- selected columns
SELECT * FROM students WHERE department = 'IT';          -- filter
SELECT * FROM students WHERE department = 'IT' AND gpa >= 3.0;
SELECT * FROM students WHERE department = 'IT' OR department = 'IM';
SELECT * FROM students ORDER BY gpa DESC;               -- **                 -- sort (ASC = smallest
first, default)
SELECT * FROM students WHERE name LIKE 'N%';             -- names starting with N
SELECT * FROM students WHERE name LIKE '%an%';           -- names containing "an"
SELECT COUNT(*) FROM students;   -- **                       -- number of rows
SELECT AVG(gpa) /* ** */ FROM students WHERE department = 'IT';    -- also MIN, MAX, SUM
SELECT department, COUNT(*) FROM students GROUP BY department;  -- **
SELECT * FROM students ORDER BY gpa DESC LIMIT 5;        -- top 5 **

-- UPDATE (always use WHERE)
UPDATE students SET department = 'IM' WHERE id = 1;
UPDATE students SET gpa = 3.6, department = 'IT' WHERE regNo = 'IT21001';

-- DELETE (always use WHERE)
DELETE FROM students WHERE id = 1;

-- Change or remove the table
ALTER TABLE students ADD COLUMN phone TEXT;
DROP TABLE IF EXISTS students;
```

CRUD	SQL	Meaning
Create	`INSERT INTO ... VALUES ...`	Add a new row
Read	`SELECT ... FROM ... WHERE ...`	Retrieve rows; WHERE filters
Update	`UPDATE ... SET ... WHERE ...`	Modify matching rows
Delete	`DELETE FROM ... WHERE ...`	Remove matching rows

Warning: `UPDATE` or `DELETE` **without WHERE** changes or deletes **every row**. In `LIKE`, `%` means “any characters”. Text values go in single quotes `'IT'`; numbers do not.

SQLite CRUD from Kotlin (lecture and lab code)

Operation	Android method
Create / INSERT	`ContentValues` + `db.insert(...)`
Read / SELECT	`db.query()` or `db.rawQuery()`, then loop through a **Cursor**
Update	`ContentValues` + `db.update(...)`
Delete	`db.delete(...)` with a selection condition

```
// INSERT
val db = DBHelper(this).writableDatabase
val values = ContentValues().apply {
    put("username", username)
    put("email", email)
    put("password", password)
}
val newRowId = db.insert("users", null, values)   // returns -1 if it failed
if (newRowId != -1L) Toast.makeText(this, "Registered!", Toast.LENGTH_SHORT).show()

// SELECT one row (login) with parameters ?
val rdb = DBHelper(this).readableDatabase
val cursor = rdb.rawQuery(
    "SELECT * FROM users WHERE email=? AND password=?",
    arrayOf(email, password)
)
if (cursor.moveToFirst()) { /* login success */ } else { /* invalid credentials */ }
cursor.close()

// SELECT all rows into a list
val c = rdb.rawQuery("SELECT * FROM users", null)
val names = mutableListOf<String>()
while (c.moveToNext()) {
    names.add(c.getString(c.getColumnIndexOrThrow("username")))
}
c.close()

// UPDATE only the password
val cv = ContentValues().apply { put("password", newPassword) }
db.update("users", cv, "email=?", arrayOf(email))

// DELETE
db.delete("users", "email=?", arrayOf(email))
```

- **Cursor:** a pointer over the result rows. `moveToFirst()` goes to the first row (false if no rows); `moveToNext()` moves row by row; `getString(columnIndex)` reads a value.
- **Use `?` placeholders** (parameterized queries), never join user input into the SQL string. This prevents **SQL injection** (a user typing SQL code into a field).
- **Close the Cursor** (`cursor.close()`) to avoid leaking resources. Use `readableDatabase` for lookups.

Problems with raw SQLite (why Room was created)

- Manual SQL queries create **more boilerplate** code.
- SQL mistakes are only found at **runtime** (the app crashes for the user).
- Cursor handling and column mapping are **repetitive**.
- Schema changes need careful **migration/version** management.
- Large codebases become harder to maintain and test.

5. Kotlin data classes **

A **data class** is a Kotlin class whose main purpose is to **hold data**. Room Entities, API response models and RecyclerView model classes are usually data classes.

```
data class Student(
    val id: Int = 0,               // default value
    val name: String,
    val regNo: String,
    var department: String,        // var = can be changed later
    val email: String? = null      // ? = nullable (may have no value)
)
```

Kotlin **automatically generates** these functions for a data class:

Generated function	What it gives you
`equals()` / `hashCode()`	Two objects with the same property values are equal
`toString()`	Readable text, for example `Student(id=0, name=Nimal, ...)`
`copy()`	A copy with some values changed: `val s2 = s1.copy(department = "IM")`
`componentN()`	Destructuring: `val (id, name) = s1`

Rules: the **primary constructor must have at least one parameter**, and every parameter must be marked `val` or `var`. `val` = read-only (cannot be reassigned), `var` = changeable.

```
val s1 = Student(name = "Nimal", regNo = "IT21001", department = "IT")
println(s1)                          // Student(id=0, name=Nimal, regNo=IT21001, ...)
val s2 = s1.copy(name = "Kasuni")
println(s1 == s2)                    // false (compares values, not memory)
```

Normal class vs data class: a normal class would print something like `Student@1b6d3586` and compare by memory address; a data class compares and prints by **values**.

6. Room

What is Room?

Room is part of **Android Jetpack**. It is an **abstraction layer over SQLite**: it does **not replace** SQLite; it still stores data in an SQLite database, but improves how the app talks to it.

Benefits:

- **SQL queries are validated at compile time** (a misspelled column name fails the build, not the user's app).
- Much **less boilerplate**: no manual Cursor handling; rows are mapped to Kotlin objects automatically.
- Clear structure: **Entity, DAO, Database**.
- Works with Kotlin **coroutines** (`suspend`) and LiveData/Flow.
- Easier migrations and testing.

Room architecture: Entity → DAO → Database

Component	Annotation	Role
Entity	`@Entity`	A **Kotlin data class** that represents a **table**; each property becomes a **column**
DAO (Data Access Object)	`@Dao`	An **interface** that defines the database **operations** (insert, query, update, delete)
Database	`@Database`	Abstract class extending **RoomDatabase**; lists the entities, holds the database and **exposes the DAOs**

The Application / ViewModel calls the **DAO** instead of writing SQL throughout the UI layer.

Entity

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val username: String,
    val email: String,
    val password: String
)
```

Annotation	Meaning
`@Entity(tableName = "users")`	Marks the class as a Room table and names it
`@PrimaryKey(autoGenerate = true)`	Unique identifier for each row; IDs created automatically
** `@ColumnInfo(name = "user_name")`	Give a column a different name from the property
** `@Ignore`	Property is not stored as a column
(no annotation)	Class properties become columns by default

DAO

```
@Dao
interface UserDao {
    @Insert
    suspend fun insertUser(user: User)

    @Update
    suspend fun updateUser(user: User)

    @Delete
    suspend fun deleteUser(user: User)

    @Query("SELECT * FROM users")
    suspend fun getAllUsers(): List<User>

    @Query("SELECT * FROM users WHERE email = :email AND password = :password")
    suspend fun login(email: String, password: String): User?     // ? = may be null (no match)

    // ** extra queries below
    @Query("SELECT * FROM users WHERE id = :id")
    suspend fun getUserById(id: Int): User?

    @Query("SELECT * FROM users WHERE username LIKE :search")
    suspend fun searchUsers(search: String): List<User>            // pass "%nim%"

    @Query("DELETE FROM users WHERE id = :id")
    suspend fun deleteById(id: Int)
}
```

DAO annotation	Generates
`@Insert`	INSERT of the object (option **: `@Insert(onConflict = OnConflictStrategy.REPLACE)`)
`@Update`	UPDATE of the row with the same primary key
`@Delete`	DELETE of that row
`@Query("SQL")`	Runs your own SQL; `:name` binds a function parameter into the query

`suspend` means the function runs inside a **coroutine** so database work happens **off the main (UI) thread**. Room does not allow database queries on the main thread, because they could freeze the screen.

Database class and instance

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}

// Build the database (do this ONCE, for example as a singleton)
val db = Room.databaseBuilder(
    applicationContext,
    AppDatabase::class.java,
    "app_database"            // database file name
).build()
```

- `entities = [...]`: the tables in this database. `version`: increase it when the schema changes and provide a **Migration** (otherwise data is lost or the app crashes).
- Build the database **once** (singleton) rather than on every screen.

Using Room (CRUD from an Activity)

CRUD	Annotation	Example call
Create	`@Insert`	`insertUser(user)`
Read	`@Query`	`getAllUsers()`
Update	`@Update`	`updateUser(user)`
Delete	`@Delete`	`deleteUser(user)`

```
lifecycleScope.launch {                      // coroutine tied to the Activity lifecycle
    db.userDao().insertUser(User(username = name, email = email, password = pass))
    val users = db.userDao().getAllUsers()
    adapter.submitList(users)                // show in RecyclerView
}
```

SQLite vs Room

Raw SQLite	Room
Direct database APIs (SQLiteOpenHelper)	Abstraction over SQLite
Manual SQL + Cursor handling	Entity + DAO + Database; automatic object mapping
SQL errors found at **runtime**	SQL **validated at compile time**
More boilerplate	Less code, cleaner, easier to maintain and test
Useful for understanding fundamentals	Preferred for modern Android apps

Key idea: Room still uses SQLite locally; it only improves how the app interacts with it.

7. Cloud databases and Firebase

Why use a cloud database?

- Local databases are tied to **one device**. A cloud database is a **shared source** of data for **many users and devices**.
- Cloud services offer real-time updates, centralised access, authentication integration and scalable backends.
- The app must handle **network availability, latency, security rules and synchronization**.

Example: a student directory in the cloud can be opened from a phone, a tablet or another authorised device.

Firebase database options

Cloud Firestore	Realtime Database
Document-oriented NoSQL database	**JSON tree**-based cloud database
Data model: **Collections** → **Documents** → **Fields**	One large JSON tree
Supports rich queries, **offline caching** and **real-time listeners**	Low-latency synchronised data and real-time state

Choose based on data model, query needs, real-time behaviour and project constraints.

Firestore data model

Term	Meaning	Example
Collection	A group of documents (like a folder / table)	`users`
Document	One record, identified by a document ID, holding fields	`user_001`
Field	A key-value pair inside a document	`name: "Nimal"`, `email: "nimal@..."`, `department: "IT"`

Documents in the same collection do not need identical fields (NoSQL, flexible schema). A document can contain sub-collections.

Firebase Android flow

Android App → **Firebase SDK** → **Cloud Database**, protected by **Security Rules**.

- The app talks through the **Firebase SDK**, not a direct database connection.
- **Authentication** and **security rules** decide who can read or write which data.

Setup steps: create a Firebase project → register the Android app (applicationId) → download **google-services.json** into the `app/` folder → add the Google Services plugin and Firestore dependency → create the Firestore database.

Firestore code

```
val db = FirebaseFirestore.getInstance()

// WRITE: add a document (Firestore creates the ID)
val student = hashMapOf(
    "name" to "Nimal",
    "registrationNo" to "IT21001234",
    "department" to "IT"
)
db.collection("students")
    .add(student)
    .addOnSuccessListener { Toast.makeText(this, "Student added", Toast.LENGTH_SHORT).show() }
    .addOnFailureListener { e -> Toast.makeText(this, "Failed: ${e.message}", Toast.LENGTH_SHORT).show()
}

// ** WRITE with your own document ID / overwrite
db.collection("students").document("student_001").set(student)

// ** UPDATE one field / DELETE a document
db.collection("students").document("student_001").update("department", "IM")
db.collection("students").document("student_001").delete()

// READ the whole collection once (lecture code)
db.collection("users")
    .get()
    .addOnSuccessListener { result ->
        for (document in result) {
            val name = document.getString("name")
            val email = document.getString("email")
            // update UI / ViewModel
        }
    }
    .addOnFailureListener { exception ->
        // handle network or permission error
    }

// QUERY: filter
db.collection("students").whereEqualTo("department", "IT").get()

// REAL-TIME: listener runs now AND every time data changes on any device
db.collection("students").addSnapshotListener { snapshot, error ->
    if (error != null) return@addSnapshotListener
    val names = snapshot?.documents?.map { it.getString("name") ?: "" }
}
```

Firestore calls are **asynchronous**: the result arrives later in the listener. Never assume a write has finished on the next line. Always handle **both** success and failure.

8. Data synchronization

Definition

Synchronization is the process of **keeping data consistent between two or more locations**. Typical Android example: **local Room database ↔ cloud database**.

Sync must handle **connectivity changes, retries, duplicate updates, stale data and conflicting edits**. The goal is not “copy everything” but clear rules for **which data is authoritative** and **when updates are exchanged**.

Source of truth: decide whether the cloud, the local database, or a defined conflict rule is authoritative.

Synchronization strategies

Strategy	How it works	Pros	Cons
Online-only	Read/write directly against the cloud	Simple architecture; always newest data	Poor or no offline experience
Periodic sync	Upload/download at intervals or when the app starts	Lower continuous network use	Data can be stale between syncs
Event-driven / Real-time	Cloud changes trigger updates to connected clients (for example Firestore snapshot listeners)	Live, collaborative data	More network/battery use
Offline-first	Local database used immediately; changes sync when connectivity returns	Fast, works without internet	Must manage pending changes and conflicts

Offline-first flow (8 steps, lecture)

1. User changes data → 2. Save locally (Room) → 3. Mark as **pending sync** → 4. Network becomes available → 5. **Push** changes to cloud → 6. **Pull** remote changes → 7. **Resolve conflicts** → 8. Update local DB

Handling conflicts

A **conflict** happens when the **same record is changed in different places before sync completes** (example: two devices edit the same student's phone number while offline).

Rule	Meaning
Last-write-wins	The most recent write is kept. Simple, but can overwrite a meaningful change
Timestamp / version-based	Each record stores a time or version number to detect newer or concurrent updates
Field-level merging	Merge independent changes to different fields of the same record
Explicit business rule	For critical data, a deliberate rule (for example "cloud always wins"), not an accident

Validation, security and reliability

- Validate before saving or syncing: required fields, email format, password rules, duplicates.
- Never trust client-side validation alone for important data.
- Use authenticated access and database security rules for cloud data.
- Show loading, success, empty, offline, permission-denied and failure states.
- Do not store plaintext passwords in production; use proper authentication (Firebase Authentication).

Recommended architecture

UI (XML/Compose) → ViewModel → Repository → Room (local DB) and Firebase (cloud)

The **Repository** coordinates the local and remote sources and gives the ViewModel **one single data API**. The UI always reads from Room; the Repository keeps Room and Firestore in sync.

Student Directory case (lecture): show at least 10 students (name, reg no, department); must work without internet; staff can add/update records. Design: **Room** stores the local copy, **Firebase** stores the shared cloud copy; when connected, fetch remote changes, update Room, and display Room data in a **RecyclerView**.

9. Comparison summary **

Feature	SharedPreferences	SQLite	Room	Cloud Firestore
Data type	Key-value	Relational tables	Relational tables (via SQLite)	NoSQL documents
Location	Device	Device	Device	Cloud server
Query language	None	SQL	SQL in `@Query`	Firestore query methods
Offline	Yes	Yes	Yes	Offline cache supported
Shared across devices	No	No	No	Yes
Error checking	None	Runtime	Compile time	Runtime
Best for	Settings, flags	Learning fundamentals, legacy	Structured local data	Shared, real-time data

SQL (relational) vs NoSQL (document): SQL uses fixed tables, rows, columns and a schema with relations; NoSQL (Firestore) stores flexible documents in collections and scales easily in the cloud.

10. Practice questions with short model answers

Why is SharedPreferences unsuitable for a large list of students? It only stores simple key-value pairs; it has no tables, relations or queries.

What is the role of SQLiteOpenHelper? It creates the database (`onCreate`) and manages version upgrades (`onUpgrade`), and gives readable/writable database access.

How are Entity, DAO and Database related in Room? The Entity defines the table, the DAO defines the operations on it, and the Database class lists the entities and provides the DAOs.

Which Room component holds the DAO instances and configures the database? The Database class (extends RoomDatabase).

Why is Room preferred over raw SQLite? Compile-time SQL checking, less boilerplate, automatic mapping to objects, cleaner and easier to maintain.

What happens if you misspell a column name in an `@Query`? The project fails to compile, so the error is caught before the app runs.

Why use `?` in a rawQuery? Parameterized queries keep user input out of the SQL string and prevent SQL injection.

Write SQL to get all IT students ordered by GPA (highest first). `SELECT * FROM students WHERE department = 'IT' ORDER BY gpa DESC;`

Write a Kotlin data class for a Book with id, title and price. `data class Book(val id: Int, val title: String, val price: Double)`

Local vs cloud storage? Local is on the device, fast and offline but not shared; cloud is remote, shared across devices, but needs a network and security rules.

What is data synchronization? Keeping data consistent between locations, such as a local Room database and a cloud database.

Why can synchronization create conflicts? The same record can be edited on two devices (or offline) before sync, so the system must decide which change wins.

Give one situation where offline-first design is valuable. A student directory or field-data app used where internet is unreliable; users can still view and edit, and changes sync later.

Explain Firestore's data model. Collections contain documents; each document has an ID and fields (key-value pairs).

Why must the app not assume a Firestore write succeeded on the next line? Firestore is asynchronous; the result arrives later in the success or failure listener.